

Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:

Студент 4 курса

Группы ИИ-26

Вирко Е.Д.

Проверил:

Андренко К.В.

Брест 2026

Цель работы: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Вариант:

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
1	MNIST	28X28	SGD

Общее задание

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST(root='./data', train=True,
                                download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False,
                                download=True, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1000,
                          shuffle=False)
```

```

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(32 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.pool(x)
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = CNN().to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

epochs = 10
train_losses = []
test_accuracies = []

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

    epoch_loss = running_loss / len(train_loader.dataset)
    train_losses.append(epoch_loss)

```

```

model.eval()
correct, total = 0, 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

acc = 100.0 * correct / total
test accuracies.append(acc)
print(f"Epoch {epoch + 1}/{epochs}  Loss: {epoch_loss:.4f}  Test
Accuracy: {acc:.2f}%")

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), train_losses, marker='o',
color='tab:blue')
plt.xlabel('Эпоха')
plt.ylabel('Ошибка (loss)')
plt.title('Изменение ошибки на обучающей выборке')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), test_accuracies, marker='s',
color='tab:green')
plt.xlabel('Эпоха')
plt.ylabel('Accuracy, %')
plt.title('Точность на тестовой выборке')
plt.grid(True)
plt.tight_layout()
plt.savefig('training_curves.png', dpi=150)
plt.show()

print(f"\nИТОГОВАЯ точность на тестовой выборке: {test_accuracies[-
1]:.2f}%")

def visualize_prediction(image_tensor, true_label=None):
    model.eval()
    with torch.no_grad():
        image_tensor = image_tensor.to(device)
        output = model(image_tensor.unsqueeze(0))
        probs = torch.softmax(output, dim=1)
        pred = torch.argmax(probs, dim=1).item()
        conf = probs[0, pred].item()

```

```

plt.figure(figsize=(4, 4))
plt.imshow(image_tensor.cpu().squeeze(), cmap='gray')
plt.axis('off')
title = f"Предсказание: {pred}  ({conf * 100:.1f}%)"
if true_label is not None:
    title += f"\nИстинный класс: {true_label}"
plt.title(title)
plt.show()
return pred

for i in range(5):
    img, lbl = test_dataset[i]
    visualize_prediction(img, lbl)

idx = np.random.randint(0, len(test_dataset))
img, lbl = test_dataset[idx]
visualize_prediction(img, lbl)

```

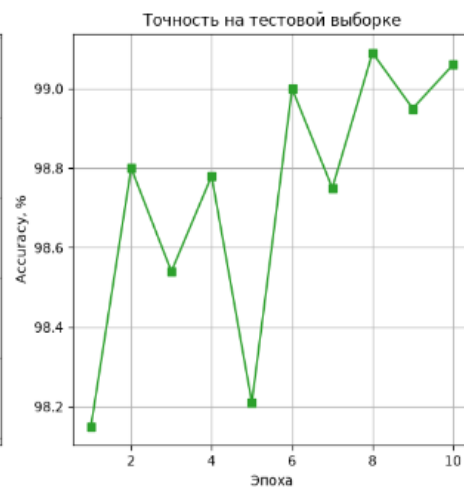
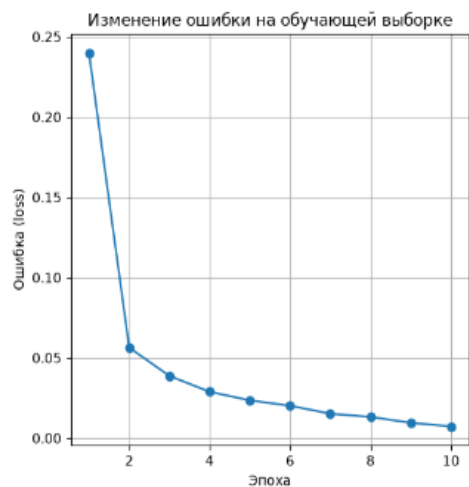
Результат программы:

```

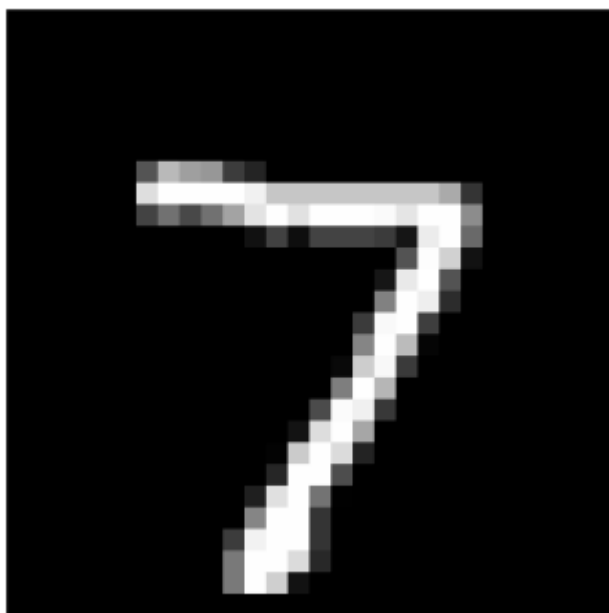
C:\Users\user\PycharmProjects\01IS1\.venv\Scripts\python.exe C:\Users\user\PycharmProjects\01IS1\main.py
Epoch 1/10 Loss: 0.2399 Test Accuracy: 98.15%
Epoch 2/10 Loss: 0.0564 Test Accuracy: 98.80%
Epoch 3/10 Loss: 0.0387 Test Accuracy: 98.54%
Epoch 4/10 Loss: 0.0289 Test Accuracy: 98.78%
Epoch 5/10 Loss: 0.0235 Test Accuracy: 98.21%
Epoch 6/10 Loss: 0.0202 Test Accuracy: 99.00%
Epoch 7/10 Loss: 0.0152 Test Accuracy: 98.75%
Epoch 8/10 Loss: 0.0132 Test Accuracy: 99.09%
Epoch 9/10 Loss: 0.0095 Test Accuracy: 98.95%
Epoch 10/10 Loss: 0.0073 Test Accuracy: 99.06%

Итоговая точность на тестовой выборке: 99.06%

```



Предсказание: 7 (100.0%)
Истинный класс: 7



Вывод: научился конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.